

CampusMatch Architecture

CampusMatch — Frontend Architecture

1. Folder Structure

2. Routing Map

3. State Management Strategy

4. Key Component Behaviors

4.1 FeedCard

4.2 Stories Bar

4.3 Surprise Flow

4.4 Post Wizard (Select → Edit → Share)

4.5 Guest Mode Gating

4.6 Bottom Nav + Post Button

5. Data Fetching Patterns

6. Performance Notes

7. PWA / Offline

CampusMatch — Frontend Architecture

Stack: React + TypeScript + Vite + Tailwind CSS, mobile-web first (390px), PWA-installable.

1. Folder Structure

```
src/
├─ main.tsx
├─ App.tsx                # Router root
├─ index.css              # Tailwind + global tokens
├─
├─ routes/                # One file per route, thin – just composes screens
│  ├─ LandingRoute.tsx
│  ├─ SignUpRoute.tsx
│  ├─ LoginRoute.tsx
│  ├─ DiscoverRoute.tsx
│  ├─ StoryPostRoute.tsx
│  ├─ StoryViewRoute.tsx
│  ├─ SurpriseRoute.tsx
│  ├─ MatchesRoute.tsx
│  ├─ ChatsRoute.tsx
│  ├─ ChatThreadRoute.tsx
│  ├─ PostRoute.tsx        # 3-step wizard, internal step state
│  ├─ ProfileMeRoute.tsx
│  ├─ ProfileUserRoute.tsx # :userId
│  ├─ SettingsRoute.tsx
│  ├─ NotificationsRoute.tsx
│  ├─ LikedPostsRoute.tsx
│  ├─ SavedSurprisesRoute.tsx
│  ├─ BlockedAccountsRoute.tsx
│  └─ ReportHistoryRoute.tsx
├─
├─ features/              # Domain-driven, not type-driven
│  ├─ auth/
│  │  ├─ components/ (OtpInput, UniversityEmailField, GuestBanner)
│  │  ├─ hooks/ (useAuth, useSignUp, useGuestMode)
│  │  └─ api/ (auth.api.ts)
│  ├─
│  ├─ discover/
│  │  ├─ components/
│  │  │  ├─ StoriesBar.tsx
│  │  │  ├─ StoryAvatar.tsx
│  │  │  ├─ FeedCard.tsx      # composes the 3 sections below
│  │  │  ├─ FeedCardHeader.tsx # avatar, name, uni badge, 3-dot menu
│  │  │  ├─ FeedCardMedia.tsx  # photo/video/gallery + video controls
│  │  │  └─ FeedCardFooter.tsx # like, view-profile link, caption
│  │  │  └─ ReportBlockSheet.tsx
│  │  └─ hooks/ (useFeed, useLikePost, useDoubleTapLike)
│  │  └─ api/ (feed.api.ts)
│  ├─
│  ├─ stories/
│  │  ├─ components/ (StoryEditor, StoryViewer, StoryProgressBar)
│  │  ├─ hooks/ (useMyStory, useStoryFeed)
│  │  └─ api/ (stories.api.ts)
│  └─
├─ surprise/
│  └─ components/
```

```

├── SurpriseWaiting.tsx
├── SurpriseCallScreen.tsx
├── SurpriseControls.tsx      # next, end, report
├── SurpriseChatOverlay.tsx
├── SurpriseFilters.tsx      # live video filter selector
├── SaveProfilePrompt.tsx
├── hooks/ (useMatchQueue, useWebRTCCall, useSignaling)
├── api/ (surprise.api.ts)

├── matches/
├── components/ (MatchCard, SuggestedCarousel)
├── hooks/ (useMatches, useSuggestions)
├── api/ (matches.api.ts)

├── chats/
├── components/ (ThreadListItem, ChatBubble, FrozenBanner, MessageInput)
├── hooks/ (useThreads, useThreadMessages, useSendMessage)
├── api/ (chats.api.ts)

├── post-creation/
├── components/
├──   ├── SelectStep.tsx
├──   ├── EditStep.tsx
├──   │   ├── FilterTab.tsx
├──   │   ├── AdjustTab.tsx
├──   │   ├── CropTab.tsx
├──   │   └── TextOverlayTab.tsx
├──   └── ShareStep.tsx      # caption + audience + submit
├── hooks/ (usePostWizard, useMediaEditor, useUpload)
├── lib/ (canvasFilters.ts, cropUtils.ts, videoTrim.ts)
├── api/ (posts.api.ts)

├── profile/
├── components/
├──   ├── ProfileHeader.tsx
├──   ├── StatsRow.tsx
├──   ├── PostsGrid.tsx      # with manage/edit/delete
├──   ├── TruncatedSection.tsx # generic "preview + View all"
├──   ├── OtherUserProfile.tsx
├──   ├── SendMessageButton.tsx
├──   └── LikeProfileButton.tsx
├── hooks/ (useMyProfile, useUserProfile, useUpdateProfile)
├── api/ (profiles.api.ts)

├── settings/
├── components/ (AccountSection, PrivacySection, BlockedList, ReportHistoryList)
├── api/ (settings.api.ts)

├── notifications/
├── components/ (NotificationItem)
├── hooks/ (useNotifications)
├── api/ (notifications.api.ts)

├── components/ui/          # Pure, reusable, no domain logic
├──   ├── BottomNav.tsx      # Discover · Matches · Post · Chats · Profile
├──   ├── PostNavButton.tsx  # raised center button, flush in nav bar
├──   ├── TopBar.tsx
├──   ├── Avatar.tsx
├──   ├── Button.tsx
├──   ├── Sheet.tsx (bottom sheet/drawer, used for 3-dot menus)
├──   ├── Modal.tsx
├──   ├── Skeleton.tsx
├──   └── EmptyState.tsx

├── lib/
├──   ├── supabaseClient.ts  # single Supabase client instance
├──   ├── queryClient.ts     # React Query client config
├──   ├── guards.ts          # RequireAuth, RequireVerified, GuestGate
├──   └── webrtc/
├──     ├── peerConnection.ts
├──     └── signalingChannel.ts

├── store/                  # Zustand stores – UI/session state only
├──   ├── sessionStore.ts    # user, isGuest, isVerified
├──   ├── postWizardStore.ts # in-progress post draft across steps
├──   └── uiStore.ts          # active sheet/modal, toasts

├── types/
├──   ├── database.types.ts  # generated from Supabase (supabase gen types)
├──   └── domain.types.ts    # FeedPost, Match, Thread, etc.

```

2. Routing Map

Path	Route	Guard
/	Landing	public
/signup	Sign Up	public
/login	Login	public
/discover	Discover (default after auth/guest)	guest-allowed (read-only)
/stories/post	Post Story	RequireAuth
/stories/:userId	View Story	guest-allowed (gated interactions)
/surprise	Surprise flow	RequireAuth + RequireVerified
/matches	Matches	RequireAuth
/chats	Chats list	RequireAuth
/chats/:threadId	Chat thread	RequireAuth
/post	Post wizard (steps as internal state, not sub-routes, to preserve draft)	RequireAuth
/profile/me	My Profile	RequireAuth
/profile/:userId	Other user's profile	guest-allowed (view), gated actions
/settings	Settings	RequireAuth
/notifications	Notifications	RequireAuth
/liked-posts	Liked posts (full list)	RequireAuth
/saved-surprises	Saved from Surprise (full list)	RequireAuth
/blocked-accounts	Blocked accounts	RequireAuth
/report-history	Report history	RequireAuth

RequireAuth redirects unauthenticated/guest users to /signup with a toast: “Sign up to continue.” RequireVerified additionally checks profile.verified === true (university email confirmed).

3. State Management Strategy

Type of state	Tool	Examples
Server/remote data	React Query (@tanstack/react-query)	feed posts, matches, chat messages, profile data
Realtime-driven data	React Query cache updated via Supabase Realtime subscription	new messages, presence, story views
Session/auth state	Zustand (sessionStorage)	current user, guest flag, verified flag
Cross-step wizard state	Zustand (postWizardStore)	selected media, applied filters/crop/text, caption, audience — persists across Select → Edit → Share without re-fetching
Ephemeral UI state	local useState / Zustand uiStore	open sheet, active tab, toasts

React Query handles caching, pagination (useInfiniteQuery for the feed), and optimistic updates (e.g., heart-like toggles instantly, rolls back on error).

4. Key Component Behaviors

4.1 FeedCard

- Three sub-sections exactly as specified: FeedCardHeader (avatar, username, uni badge, 3-dot menu → ReportBlockSheet), FeedCardMedia (photo/video/gallery, video gets play/pause/mute controls, gallery gets swipe + dot indicator), FeedCardFooter (heart, “View profile” link, optional caption).
- Video autoplay is driven by an IntersectionObserver hook (useInViewAutoplay) — only the card mostly in viewport plays, muted by default.
- Double-tap-to-like uses a tap-timing hook (useDoubleTapLike) layered over the media element.

4.2 Stories Bar

- Horizontal scroll, first item is “Your Story” (+ icon if none posted, ring-progress avatar if posted). Tapping own avatar with an existing story opens viewer on self; tapping “+” opens StoryPostRoute .
- StoryViewer is a full-screen takeover with per-story progress bars, tap-left/right to navigate, auto-advance timer.

4.3 Surprise Flow

- `useMatchQueue` writes presence to a `surprise_queue` table/channel, listens for a paired match event from an Edge Function, then hands off to `useWebRTCCall`.
- `useWebRTCCall` owns the `RTCPeerConnection`, exchanges SDP/ICE over a Supabase Realtime channel scoped to the call's session id (`useSignaling`).
- Filters during the call are applied client-side via CSS filters or a `<canvas>` overlay on the local video track — cosmetic only, not sent over the wire unless burned into the outgoing track via `canvas.captureStream()`.
- “Next” tears down the current peer connection and re-enters the queue. “End” exits to `SaveProfilePrompt`.

4.4 Post Wizard (Select → Edit → Share)

- Single route, three internal steps tracked in `postWizardStore`, not separate URL routes — this keeps the in-progress media/edits in memory without re-selection on back-navigation.
- `EditStep` tabs (Filters/Adjust/Crop/Text) all write into the same draft object; preview re-renders live via CSS filters + canvas composition.
- On Share, `useUpload` compresses/exports the final media (canvas → blob), uploads to Supabase Storage, then inserts the post row with the chosen audience scope.

4.5 Guest Mode Gating

- `GuestGate` wraps interactive controls (like button, send message, post button, surprise button, story-post button). On tap, if `sessionStore.isGuest`, it intercepts the click, shows a toast/modal “Sign up to continue,” and routes to `/signup` instead of executing the action.
- Discover feed itself renders fully for guests (read access only); everything else behind `RequireAuth` simply redirects.

4.6 Bottom Nav + Post Button

- `BottomNav` renders five slots; the center slot is `PostNavButton`, styled as a raised gradient square but constrained within the nav bar's height (no overflow/negative margin) per the latest UI decision.

5. Data Fetching Patterns

- All Supabase calls go through `features/*/api/*.api.ts` — components never call `supabaseClient` directly. This keeps query logic testable and swappable.
- React Query keys follow a consistent convention: `['feed', cursor]`, `['thread', threadId]`, `['profile', userId]`, `['matches', userId]`, enabling targeted invalidation (e.g., sending a message invalidates `['thread', threadId]` and `['threads']`).
- Realtime subscriptions (chat messages, notifications, presence) are set up in the relevant hook (`useThreadMessages`, `useNotifications`) and write directly into the React Query cache via `queryClient.setQueryData` on incoming events, avoiding refetch storms.

6. Performance Notes

- Feed and grids use `IntersectionObserver`-based lazy loading for images/videos.
- Cursor-based pagination (`created_at + id`) for feed, matches, chats, notifications.
- Heavy editing (filters/crop/text) done on a single `<canvas>`, not re-rendered per slider tick beyond `requestAnimationFrame` throttling.
- Route-level code splitting via `React.lazy` for Surprise (WebRTC libs) and Post wizard (canvas/editing libs) since they're the heaviest bundles.

7. PWA / Offline

- `vite-plugin-pwa` for service worker + manifest.
- Cache-first for app shell, network-first for feed/chat data.
- Local persistence of `sessionStore` (guest flag, last route) via `localStorage` so a refresh doesn't bounce a guest back to Landing unnecessarily.